

One Person Lab

One Person Lab 白皮书

让复杂知识工作进入可交付、可复查、可持续推进的专业系统

One Person Lab 的核心承诺，是把 AI 专家能力从一次问答变成可阶段推进、可复查、可交付的专业工作系统；Framework 管运行纪律，App / Cloud 管用户工作面，Foundry Agents 管领域质量与交付权威。

OPL Framework / One Person Lab App / OPL Cloud / Foundry Agents

2026-07-04

Public whitepaper

目录

1	定位摘要	2
2	为什么要处理这件事	2
3	OPL 的答案：让 AI 像专业团队一样推进工作	2
4	这套设计从哪里来	3
5	认知计算：阶段内开放式判断	3
6	OPL 的三层产品模型	3
6.1	OPL Framework	4
6.2	One Person Lab App	4
6.3	Foundry Agents	4
7	Framework、Cloud 与 Foundry Agents 的关系	4
8	设计原则：让 AI 做专家工作，让系统守住边界	4
8.1	AI-first, contract-light	5
8.2	交付即推进	5
8.3	目标先于路径	5
8.4	真相归主	5
8.5	抓大放小	5
9	OPL 背后的十类能力	6
9.1	OPL Charter	6
9.2	OPL Atlas	6
9.3	OPL Workspace	6
9.4	OPL Pack	6
9.5	OPL Stagecraft	6
9.6	OPL Runway	7
9.7	OPL Ledger	7
9.8	OPL Console	7
9.9	OPL Foundry Lab	7
9.10	OPL Connect	7
10	专业智能体：同一工作方式，不同专业场景	7
10.1	Med Auto Science	8
10.2	Med Auto Grant	8
10.3	RedCube AI	8
10.4	OPL Meta Agent	8
10.5	OPL Book Forge	8
10.6	未来 Foundries	8
11	为什么 OPL 能稳定运行	9
12	用户会如何感知 OPL	9
13	为什么用户可以相信 OPL 专业	9
14	结语	10

发布日期：2026-07-04

适用对象：希望理解 OPL 能解决什么问题、怎样推进复杂工作、以及专业智能体如何协作的用户、合作者、早期采用者和技术决策者。

核心判断：One Person Lab 的核心承诺，是把 AI 专家能力从一次问答变成可阶段推进、可复查、可交付的专业工作系统；Framework 管运行纪律，App / Cloud 管用户工作面，Foundry Agents 管领域质量与交付权威。

1 定位摘要

- One Person Lab 面向复杂知识工作的持续交付，让论文、基金、汇报、书籍、专利、答辩、审稿等工作以阶段方式推进到成果交付。
- OPL Framework 是 One Person Lab 的运行底座，围绕阶段、工作空间、文件、证据、恢复和交接设计。
- One Person Lab App 是用户工作台，负责把运行框架和专业智能体变成可见、可操作、可检查的产品体验。
- OPL Cloud 是 App、在线 Workspace、Gateway、Console、托管资源和证据链的云端产品包装；它消费 Framework 能力，但不改变 Framework 的源码模块和领域权威边界。
- 专业智能体负责具体领域，例如研究、基金、汇报、书籍和智能体构建；专利、报奖、学位论文、审稿等方向会在边界成熟后继续加入。
- OPL 的设计从用户要交付的成果反推：用户要的是可靠推进，系统就必须说清楚材料在哪、做到哪、生成了什么、下一步由谁处理。

2 为什么要处理这件事

AI 已经很擅长回答一个问题、生成一段代码或润色一份材料。当任务进入论文、基金、汇报、专利、答辩和审稿这类长周期成果时，用户关注的是整项工作如何持续推进到交付。

一个人做研究、写基金、准备汇报、整理专利或推进答辩材料时，材料很多，工作跨越很多天甚至很多周，中间会反复修改、审阅、补证据、换方向、重开上下文。多轮推进之后，用户需要清楚知道当前阶段、最新版文件、已经形成的判断和待处理卡点。

项目管理工具擅长记录任务名和状态；固定自动化擅长执行确定步骤；AI 对话擅长生成内容。长周期知识工作还需要一套能承载阶段、证据、文件、负责人和交付边界的运行结构，让理解、比较、创作、审阅和修订可以在同一工作线上持续发生。

OPL 为高价值知识工作提供这样的操作系统：一个人也可以拥有类似专业团队的持续推进能力，AI 围绕阶段目标组织材料、工具和候选方案，系统把复杂成果一步步推到可以检查、可以修改、可以交付的位置。

- 多轮推进之后，当前阶段如何持续记录？
- 材料、文件、修改和证据如何形成可回溯的文件历史？
- 准备、执行、审核、修订和交付如何保持清楚边界？
- 用户离开电脑后，任务如何保留运行线索，并在回来时呈现进展、卡点和下一步？
- 多个专业 Agent 如何共享一套运行、文件、进度和交付体系？

3 OPL 的答案：让 AI 像专业团队一样推进工作

One Person Lab 正是围绕这些问题设计的。它把复杂知识工作拆成一个个能推进的阶段：准备材料、执行创作、独立审阅、修订完善、交付收口。

每个阶段都围绕一个真实成果增量工作。AI 可以在同一阶段里整理资料、提出候选方案、比较取舍、调用工具、接受审阅并继续修订。用户通过工作台看到进度、文件、证据、卡点和下一步都被清楚保留下来。

对话工具擅长处理当次问题；OPL 把关注点推进到整项工作的阶段、产物、确认记录、负责人和工作空间。用户对工作的信任来自可检查的阶段记录和交付物。

在这个模型里，AI 保持专家判断空间，可以理解材料、比较方案、创造内容、接受反馈、继续修订。系统负责让这些开放式工作具备边界、记录和可恢复落点。

- 用户看到任务进展、当前文件和交付物。

- Agent 接收阶段目标、材料上下文和可用能力。
- 系统保存证据、确认记录、卡点和恢复点。

4 这套设计从哪里来

OPL 的当前品牌模块来自长周期知识工作的真实摩擦：一次性回答需要延伸为持续交付，自动化脚本需要连接专家判断，界面可见性需要连接质量完成。

第一层来源，是 OPL 自己要服务的任务：论文、基金、汇报、专利、报奖、学位论文、审稿和修回。这些任务都有明确交付物，也都有大量中间判断。它们要求 AI 能做专业工作，同时要求系统能说清楚当前阶段、材料来源、文件位置、审阅结果、卡住原因和下一步。

第二层来源，是 MAS、MAG、RCA、Book Forge 和 OPL Meta Agent 等 Foundry Agents 的真实实践。医学研究需要证据包、稿件、发表质量检查；基金写作需要方向判断、申请书结构、模拟评审和修订；视觉交付需要故事线、渲染、审阅和导出；书籍写作需要大纲、章节、风格和出版交接；智能体构建需要脚手架、测试接管、机制改进和回滚。不同领域的内容不同，但它们都需要同一类运行外围：阶段、工作空间、确认记录、卡点、文件历史、进度展示和恢复机制。

第三层来源，是成熟工程系统的分层经验。OPL 借鉴能力目录、可恢复执行、维护者工作台、文件历史、机器可读描述和架构决策记录等治理思想，并把这些原则落到 One Person Lab 自己的用户路径、专业智能体和交付边界里。

- 从用户痛点来：复杂成果从一次回答延伸为持续推进。
- 从领域实践来：MAS、MAG、RCA、Book Forge 和 OPL Meta Agent 证明不同专业 agent 需要共享运行骨架。
- 从工程经验来：可发现、可恢复、可检查、可派生的系统才容易长期维护。

5 认知计算：阶段内开放式判断

固定自动化流程擅长处理确定步骤：先做 A，再做 B，最后输出 C。复杂知识工作需要更强的阶段内判断：写一篇文章、改一个基金本子、做一套正式汇报，过程中经常需要反复判断、比较、推翻、重写和审阅。

OPL 把这种阶段内的开放式专家工作称为认知计算：AI 在一个可观察、可接力的阶段里完成理解、比较、创作、审阅和修订。系统提供目标、材料、工具边界、权限、质量检查和交接要求，让执行者在阶段内自主组织工作。

这样设计的好处是，AI 执行者越强，OPL 的阶段能力就越强。模型升级、提示词改进、技能改进、领域知识改进，都能直接提升阶段内的真实工作能力；同时，OPL 的阶段、确认记录、工作空间、进度展示和恢复机制让这些能力转化为可追踪的持续工作。

用户最终看到的是工作在往前走：下一版文件形成了，证据能解释，审阅有边界，卡点能被看见，交接能继续。

- OPL 保留专家工作所需的开放判断空间。
- OPL 让 AI 在阶段内有足够空间完成真实判断。
- OPL 用系统边界保证开放式工作可见、可复核、可恢复。

6 OPL 的三层产品模型

OPL 对外呈现为一个统一品牌，对内保持三层清楚分工：Framework、App、Foundry Agents。这样的结构让用户体验、运行框架和专业智能体可以各自演进，并通过共同约定保持一致。

默认运行链路

```

用户 / App / CLI
  -> Codex CLI 标准执行者
    -> OPL 显式激活
      -> 可恢复的阶段运行
        -> 选定的 Foundry Agent
          -> 领域事实 / 质量检查 / 交付物
  
```

6.1 OPL Framework

运行框架

Framework 是让复杂工作能跑起来并持续跑下去的底座。它负责执行会话、显式激活、阶段控制、任务队列、长跑运行、共享约定、恢复和检查。它让任务可以被启动、恢复、展示和收口。

- 默认第一公民执行者是 Codex CLI。
- Temporal-backed provider 是生产级在线运行的必需底座。
- Framework 负责通用运行外围，领域事实、质量结论和产物权威由对应 Foundry Agent 持有。

6.2 One Person Lab App

用户工作台

App 是 OPL 的产品入口。它把运行状态、工作空间和 Foundry Agent 进度组织成可操作工作台：选择任务、查看进度、打开文件、处理卡点、接收更新。当前普通用户形态是 OPL-branded App shell：用户面对的是 OPL 任务和 Foundry Agent 入口，而不是底层技术后端或执行者选择器。

- App 固定 Codex CLI 作为标准执行路径。
- App 使用 Framework 和领域智能体提供的状态，不复制运行系统或领域真相。
- App 负责界面产品事实、发布门、用户教程和可用性验证。
- App 负责展示和交互，质量完成结论回到对应领域智能体。

6.3 Foundry Agents

领域智能体

Foundry Agents 是 MAS、MAG、RCA、OPL Meta Agent、OPL Book Forge 以及未来 Patent、Award、Thesis、Review 等领域智能体。它们像专业团队里的不同角色：研究、基金、视觉交付、智能体构建和书籍写作各自有材料理解、审阅标准和交付边界。它们以统一 OPL 结构运行，并保留自己的领域判断、质量检查、产物权威和直接使用路径。

- 同一生命周期：材料进入、阶段执行、质量检查、负责人确认、交付物交接。
- 不同领域保留不同输入、输出、知识、评价标准和权威动作。
- OPL 通过引用展示领域状态，领域正文和产物内容由对应智能体管理。

7 Framework、Cloud 与 Foundry Agents 的关系

OPL Framework 和 OPL Cloud 不是两套叙事。Framework 定义复杂知识工作如何被阶段化、如何留下证据、如何恢复、如何交接，以及不同 Foundry Agent 如何保留自己的领域权威。Cloud 把这套工作方式扩展成在线产品体验：用户可以通过 OPL App 或在线 OPL Workspace 进入同一条工作线，通过 OPL Gateway 获得 AI 能力，通过 OPL Fabric 使用计算、存储、环境和连接器，通过 OPL Ledger 留下回执，通过 OPL Console 管理组织、权限、套餐和资源策略。

这种分工让 OPL 同时保持专业和克制。Framework 不把云端产品包装反向写成新的源码模块；Cloud 不复制 Framework 的运行真相，也不替代 Foundry Agent 的质量判断。用户看到的是一个连续的 One Person Lab：本机能跑、云端能接、团队能管、结果能复查。

- Framework 解决“复杂工作怎样可靠推进”。
- Cloud 解决“这些推进能力怎样在线化、组织化、资源化和可计量”。
- Foundry Agents 解决“具体专业领域怎样判断质量并交付成果”。

这也是 OPL 的核心特色：不是把 AI、云资源和任务管理简单拼在一起，而是让它们围绕正式成果交付形成一条清楚的责任链。

8 设计原则：让 AI 做专家工作，让系统守住边界

OPL 的设计原则不是一组抽象口号，而是一套面向复杂交付的品牌判断：把开放式专家工作交给 AI，把安全、权限、确认记录和恢复交给系统；阶段一旦形成可接力成果，就继续向下推进。用户要看到的是成果在变好、责任在流转、

卡点能解释、下一步能继续。

8.1 AI-first, contract-light

AI 做专家工作，合同守住下限

复杂知识工作交给 AI 执行者做开放式判断、写作、评审和修订。合同只负责身份、权限、安全动作、确认记录、卡点、检查、恢复和状态展示，不把工具顺序、创作策略或评审方法写死。这样模型、提示词、技能和知识面进步时，OPL 的真实工作能力也会自然变强。

- 当前默认且第一公民执行者是 Codex CLI，其他执行者必须显式接入并留下确认记录和检查线索。
- 合同保护系统下限，AI 保留阶段内专家判断空间。

8.2 交付即推进

Deliver, then advance

OPL 以阶段作为可观察、可恢复、可检查的工作单元。一个阶段只要形成可接力成果、负责人确认、清楚卡点或明确人工决策，就应该进入下一步，而不是停在反复预检、状态解释或等待谁再判断能不能继续。

- 真实进度按成果、证据、决策、交接和可接力增量计算。
- 质量检查用于发现差距和决定下一次修订方向，不把普通推进困在诊断循环里。

8.3 目标先于路径

Purpose first

OPL 先问用户要交付什么：论文、基金、汇报、书稿、专利，还是一个可接力的智能体。目录、状态、运行系统、队列和验证都服务这个目的；现有实现、工具偏好和中间指标不能反过来定义用户路径。

- 用户目标决定系统形状，不让历史实现反向定义长期设计。
- 不能产出成果、责任方、证据或交接的设计面，应收薄、下沉或归档。

8.4 真相归主

Truth stays with the owner

Framework、App 和 Foundry Agents 各有自己的权威边界。OPL 管通用运行外围，领域智能体管领域事实、质量检查和产物权威，App 管用户界面和发布体验。界面、运行框架和状态展示可以使用领域信号，但不能替领域负责人签发最终判断。

- 每类事实都有单一责任方，避免长出第二真相源。
- 论文质量、基金可资助性、视觉交付和书稿完成度，必须回到对应领域智能体或独立审阅环节。

8.5 抓大放小

Guard the essentials

OPL 只把启动安全、越权、关键运行事件、阶段组合关系、硬性人工确认、执行者绑定、不可逆权限和负责人确认这类大边界设为硬门。提示词、技能、工具、知识、评分标准、容量、监控、假设、回放和诊断证据默认进入建议、检查、清理或后置证据，不让细节反向卡住普通推进。

- 抓大保证系统不越权、不误闭合、不制造第二真相源。
- 放小让任务先形成可审阅、可接力、可验证的下一步结果。

9 OPL 背后的十类能力

当前十个模块是 OPL Framework 的能力分区，而不是模块数量上限。它们把顶层理念落到真实运行：先建立语言和边界，再组织能力目录和工作空间，再定义专业智能体怎样接入，最后支撑阶段执行、长跑恢复、证据记录、用户控制台、智能体工坊和外部连接。新的能力只有在责任、目的、边界和验收方式都说清楚后，才会进入这套结构。

9.1 OPL Charter

语言和边界

Charter 固定 OPL 的顶层定位、命名、产品层级、ADR/RFC 和品牌组合治理。它回答什么叫 OPL、为什么存在、什么属于 Framework、什么属于 App、什么属于 Foundry Agent。Charter 让长期演进始终沿着同一套语言展开。

- 让整个系统拥有统一语言。
- 让路线演进、命名和品牌表达保持一致。

9.2 OPL Atlas

可发现目录

Atlas 管理智能体、能力、入口、责任方、依赖和生命周期目录。用户需要知道 OPL 能做什么，系统需要知道能力来自哪里、谁负责、如何调用、当前状态是什么。Atlas 把 Foundry Agents、能力包、工具卡和公开入口放进可发现目录。

- 负责能力发现和目录组织。
- 把领域描述、模块注册和公开入口放进统一目录。

9.3 OPL Workspace

用户和 Agent 共同检查的项目空间

Workspace 把材料、共享资源、阶段输出、交接和文件生命周期组织成可检查结构。复杂任务最终必须落到文件、材料和交付物上；用户能在项目空间里看到当前文件和最终交付物。

- 默认模型是 Workspace Group -> Project Unit -> Stage Artifact Unit。
- 真实完成绑定文件清单、确认记录和当前指针。

9.4 OPL Pack

专业智能体接入方式

Pack 定义 Foundry Agent 如何声明自己的专业能力、阶段动作、权威函数、能力目录、生成入口和对齐状态。它回答一个领域智能体怎样被 OPL 标准化接入，同时不让 OPL 接管领域处理逻辑、负责人确认或质量裁决。

- 固定 domain pack、capability registry、authority ABI 和 generated surface 输入。
- 让标准智能体更容易生成、验证、分发和维护。

9.5 OPL Stagecraft

阶段内认知计算设计

Stagecraft 负责阶段设计、提示词、技能、工具可用性、知识、评价标准、能力调用策略和独立质量检查。它回答这个阶段要让 AI 怎样像专家一样工作，并在保留开放式专家空间的同时，让输入、输出和边界可检查。像 MAS Scholar Skills 这样的能力包，进入这里作为可复用专业能力，而不是新的领域真相源。

- 工具目录说明可用能力、权限和边界。
- 复杂质量判断进入独立阶段或独立审阅环节。

9.6 OPL Runway

可恢复的长跑执行

Runway 负责持久执行、类型化队列、尝试记录、租约、重试、唤醒和人工确认。它回答这项工作怎么持续跑下去。长期任务通过 Runway 保留运行线索、恢复点和人工介入边界。

- 承接可恢复的阶段运行。
- 承载长跑执行和恢复语义，领域结论回到对应 agent。

9.7 OPL Ledger

证据、确认记录和文件历史

Ledger 保存证据引用、确认记录、清楚卡点、文件历史、恢复证明和来源记录。它回答为什么我们相信当前状态。任务为什么可以继续、为什么卡住、如何恢复，都有可追踪依据。

- 保存引用和文件历史，让证据可追踪。
- 领域正文、记忆和产物内容保留在对应智能体的权威边界内。

9.8 OPL Console

用户和维护者的可见控制台

Console 把就绪状态、当前责任方、下一步动作、卡点、产物和展开详情展示到 App 和维护者工作台。它回答用户现在应该看什么、做什么，并把运行状态转化为可行动信息。

- 默认优先呈现责任方变化。
- 使用进度视图，并把运行状态呈现为工作台视图。

9.9 OPL Foundry Lab

智能体创建与机制改进

Foundry Lab 支撑新智能体创建、测试接管、机制改进、试运行、升级和回滚。它回答 OPL 怎样持续变强。运行证据可以转成可执行改进任务，帮助智能体机制持续演进。

- 用于改进智能体机制。
- 与 MAS、MAG、RCA 或未来智能体的领域权威保持清楚分工。

9.10 OPL Connect

外部调用和分发连接

Connect 把同一套能力说明连接到 CLI、MCP、OpenAI/AI SDK tools、Skill/plugin、模块安装、package channel、发布/安装分发和漂移检查。它回答同一套能力怎样被安装、发现、调用和分享，同时避免把每个 CLI 命令平铺成新的工具真相源。

- 让能力可以被安装、同步、发现和调用。
- 保持外部调用与领域自有语义一致。

10 专业智能体：同一工作方式，不同专业场景

Foundry Agents 让 OPL 的品牌进入具体专业场景：研究、基金、视觉交付、书籍写作、智能体构建，以及未来的专利、报奖、学位论文和审稿。每个智能体都面向一个高价值工作流，拥有自己的领域知识、质量检查和交付权威，同时共享 OPL 的阶段式运行骨架。

10.1 Med Auto Science

Research Foundry

面向医学和科研论文 workflow，从数据、文献、研究问题、分析、证据包到稿件和投稿包。研究工作特别需要证据、审阅、修订和投稿边界，所以 MAS 持有研究事实、发表质量检查、产物权威和负责人确认。

- 用户感知：把研究项目持续推进到可审阅论文和投稿材料。
- OPL 承载阶段尝试、队列、确认记录展示和工作空间/运行支撑。

10.2 Med Auto Grant

Grant Foundry

面向基金方向判断、申请书写作、作者侧模拟评审和修订。基金工作需要重新组织问题、创新点、技术路线和评审说服力；MAG 复用研究证据和记忆，同时保持基金事实与可资助性评审边界。

- 用户感知：从想法到申请书草稿、模拟评审包和修订计划。
- 质量判断回到 MAG 自己的质量检查和负责人确认。

10.3 RedCube AI

Presentation Foundry

面向视觉交付、PPT、报告、storyboard、render、review 和 export。汇报材料需要叙事、结构、视觉和可展示文件同时成立；RCA 持有视觉事实、审阅/导出结论和产物权威。

- 用户感知：把材料转成可展示的视觉交付物。
- OPL 负责阶段文件夹、产物清单、确认记录引用和进度展示。

10.4 OPL Meta Agent

Foundry Lab managed module

面向智能体创建、测试接管、机制改进和目标智能体工单。它帮助 OPL family 更系统地生产智能体、测试智能体，并从失败中改进机制；领域智能体的最终事实仍由对应领域责任方持有。

- 用户感知：OPL 能持续变强、持续生成和修复自己的 agents。
- 边界：生成改进工单或清楚卡点，领域责任方负责最终签收。

10.5 OPL Book Forge

Book Foundry

面向书籍、长篇手稿、章节结构、风格控制、参考材料整合和出版交接。书籍写作需要长期上下文、章节级推进、版本控制、审阅和导出边界；Book Forge 持有书稿事实、质量/导出判断和 manuscript package 交接权威。

- 用户感知：把想法、材料和章节推进成可审阅、可导出的书稿包。
- OPL 负责运行骨架、阶段展示、文件历史和托管入口。

10.6 未来 Foundries

IP / Award / Thesis / Review

IP、Award、Thesis、Review 等工作流会沿用同一骨架：材料进入、领域包解释、阶段式执行、独立质量检查、负责人确认或清楚卡点、交付物交接、OPL 展示。它们会在对应领域边界、质量检查和交付权威成熟后成为新的 Foundry。

- 每个新智能体都有自己的领域事实和质量边界。
- 共享 OPL 运行框架，并把领域差异保留在对应 Foundry 内。

11 为什么 OPL 能稳定运行

OPL 的稳定感来自一组可靠边界：开放式智能工作在阶段内发生，系统负责留痕、交接、恢复和质量边界。它让 AI 像专家一样工作，也让系统像专业组织一样运行。

第一，任务以阶段为单位推进。一个阶段足够大，可以让 AI 执行者完成真实工作包；同时也足够清楚，可以定义输入、输出、权限、质量检查和收口形态。

第二，责任边界清楚。Framework 持有框架事实；App 持有产品和展示事实；Foundry Agent 持有领域事实、质量结论和产物权威。每个判断都有归属，展示、运行和质量结论保持清楚分工。

第三，证据以引用和文件历史方式保存。OPL 记录确认记录、清楚卡点、文件历史、当前指针和恢复证明，让任务可以被检查和恢复；领域正文、产物内容和质量裁决保留在对应责任方处。

第四，用户默认看到下一步动作。工作台优先回答当前等待谁、缺什么、下一步能做什么，再提供事件记录、计数和诊断详情。

第五，系统从单一来源派生多个入口。CLI、App、Skill/plugin、MCP、OpenAI/AI SDK tool、package channel 和 release/install 分发都围绕同一套能力说明生成，避免每个入口各说各话。

第六，设计始终从用户要交付的成果反推。默认入口聚焦进度、产物、证据、卡点、责任方和下一步；诊断细节进入展开详情和维护面。这样的减法治理让 OPL 保持清楚、专业和可维护。

- 这就是 OPL 的核心体验：AI 有足够自由度完成专家工作，系统有足够纪律保证工作可见、可复核、可恢复、可交付。
- 用户对工作的信任来自阶段、产物、确认记录、责任方和工作空间。

12 用户会如何感知 OPL

OPL 的用户体验目标，是让复杂知识工作呈现为一条可以持续推进、可以检查、可以交接的工作线。

在 App 中，用户选择任务和 Foundry Agent。标准路径使用 Codex CLI 执行者；运行状态被组织成就绪状态、当前责任方、下一步动作和展开详情。普通用户不需要选择底层执行者、技术后端或 shell 实现；这些差异留在开发者和维护者诊断面。

在工作空间中，用户看到材料、阶段输出、交付物和当前指针。运行状态解释工作如何推进，项目文件承载可检查、可复制、可分享和可归档的交付物。

在长期任务中，用户看到每个阶段的进度、卡点和交接。任务可能需要人类批准、补材料、接受路线调整或等待领域负责人确认，但这些状态都会显式出现。

用户从任务入口进入，查看当前阶段，打开产物，处理必要卡点，并在关键节点做判断。复杂运行细节由系统组织，面向用户呈现为清楚的进度、文件、证据和下一步。

- OPL 把复杂性收进系统，把可行动信息呈现给用户。
- OPL 把 AI 的创造力留在阶段内，把责任和证据留在系统边界上。
- OPL 把一次性助手升级成可以持续运行的实验室伙伴。

13 为什么用户可以相信 OPL 专业

专业感不是来自更多按钮、更多后端或更复杂的配置，而是来自系统能持续回答几个关键问题：这项工作要交付什么，当前推进到哪里，谁负责下一步，证据在哪里，什么判断不能越权，卡住后怎样恢复。

OPL Framework 把这些问题放进默认结构。阶段让工作有边界；Workspace 让材料和产物有位置；Ledger 让证据和确认记录有出处；Console 让责任方和下一步可见；Runway 让长期任务可恢复；Pack 和 Stagecraft 让专业智能体以统一方式接入，同时保留自己的领域判断。

这种专业性也体现在 OPL 不伪造结论。Framework 可以说明运行、记录、恢复和投影是否成立，但论文质量、基金可资助性、视觉交付、书稿完成度和生产级发布，必须回到对应 Foundry Agent、App release owner 或真实运行证据。该说“已形成可接力成果”时不包装成“生产就绪”；该说“等待领域负责人判断”时不包装成“系统已完成”。

用户因此获得的不是一个会说漂亮话的助手，而是一套能持续推进复杂成果、保留判断依据、暴露卡点并支持后续接力的专业工作系统。

14 结语

One Person Lab 的长期目标，是让一个人也能拥有专业团队级别的复杂知识工作推进能力。AI 负责理解、比较、创作、审阅和修订；Framework 负责阶段、证据、恢复、边界和交接；App 与 Cloud 负责把这些能力变成用户可见、可管理、可继续的工作体验；Foundry Agents 负责具体领域的质量判断和交付权威。

这就是 One Person Lab 的判断：下一代 AI 产品的价值，不只在于回答得更快，而在于能把正式工作持续推进到用户可以检查、可以相信、可以交付的位置。